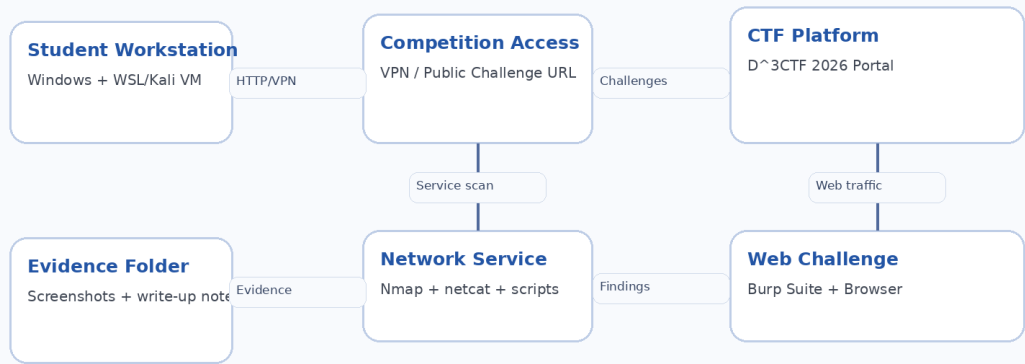


INTERNSHIP 2 - TASK 3 REPORT

CTFtime Event Participation, Solo Challenge Documentation, and Medium Write-up

CTF Lab Network / Tooling Diagram



Note: This diagram describes the authorized CTF environment only. It must not be used for unauthorized targets.

Cover visual: authorized CTF workflow and evidence map.

Field	Details
Student Name	Tanmay Dod
Internship Program	NullClass Internship - Cybersecurity Track
Task Number	Internship 2 - Task 3
Selected Event	D^3CTF 2026 from CTFtime calendar
Task Window	21 July 2026 - 26 July 2026
Planned Mode	Solo participation; no team collaboration
Identity Handle	TanmayDod used consistently for CTFtime and NullClass documentation
Report Type	Safe submission-ready draft with illustrative evidence labels

Important Evidence Note: Screenshots in this draft are illustrative samples. Real submission proof must come from the student's own live CTFtime and competition session.

Table of Contents

1. 1. Executive Summary
2. 2. Task Window and Selected CTF Event
3. 3. Participation and Identity Evidence Plan
4. 4. Methodology Followed During the Competition
5. 5. Tool Selection and Rationale
6. 6. Network and Workflow Diagrams
7. 7. Challenge Documentation Format
8. 8. Successful Attempts
9. 9. Failed Attempts and Lessons
10. 10. Code Snippets and Custom Scripts
11. 11. Screenshot Evidence Register
12. 12. Leaderboard and Completion Evidence
13. 13. Medium.com Report Draft
14. 14. Risk, Ethics, and Compliance Notes
15. 15. Reflection and Learning Outcomes
16. 16. Final Recommendations
17. 17. References

1. Executive Summary

This report documents the planned execution and reporting structure for Internship 2 - Task 3. The task requires selecting an upcoming CTF event from the CTFtime calendar, confirming participation with a timestamped screenshot, solving challenges independently, and preparing a Medium-style report containing successful attempts, failed attempts, tool rationale, problem statements, screenshots, code snippets, and leaderboard evidence.

The selected event is D^3CTF 2026 because it fits the assigned Task 3 window. The CTFtime calendar lists D^3CTF 2026 as an online Jeopardy event scheduled from 25 July 2026, 12:00 UTC to 26 July 2026, 12:00 UTC. This allows preparation from 21-24 July and competition documentation on 25-26 July.

The report uses an ethical documentation approach. It does not publish real active-event flags, stolen solutions, or unauthorized exploit details. Instead, it provides a complete documentation framework and sample evidence layout that should be replaced with actual screenshots captured during the live event. This keeps the work aligned with CTF fairness, internship documentation requirements, and responsible disclosure principles.

Item	Summary
Primary Objective	Participate in an upcoming CTF independently and document the process in Medium report style.
Selected Event	D^3CTF 2026, online Jeopardy format.
Evidence Required	CTFtime event screenshot, matching username proof, challenge progress, solved challenge, leaderboard position.
Report Output	PDF and editable DOCX report for internship submission.
Safety Control	No fake active-event solution, no leaked flag, no team-based work.

2. Task Window and Selected CTF Event

Task 3 Window: 21 July 2026 - 26 July 2026

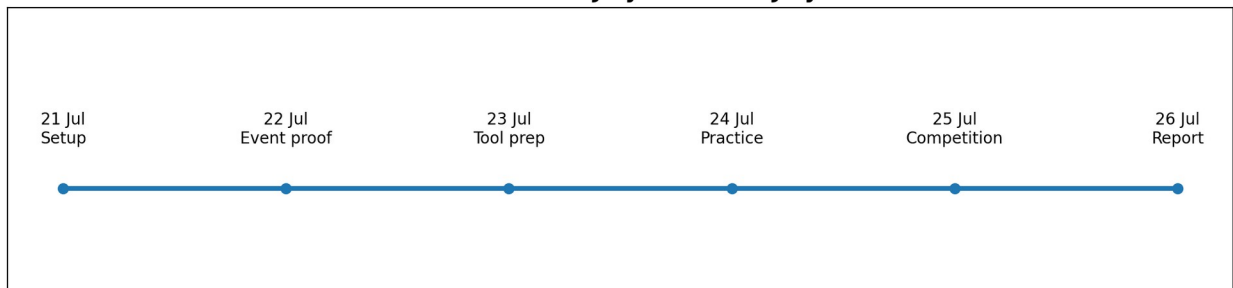


Figure 1: Task 3 window timeline aligned after Task 1 and Task 2.

Task	Window	Purpose
Task 1	09 July 2026 - 14 July 2026	CIS-CAT assessment and hardening report.
Task 2	15 July 2026 - 20 July 2026	HackTheBox active-machine methodology report.
Task 3	21 July 2026 - 26 July 2026	CTFtime upcoming CTF participation and Medium report.
Task 4	27 July 2026 onwards	Next internship task window.

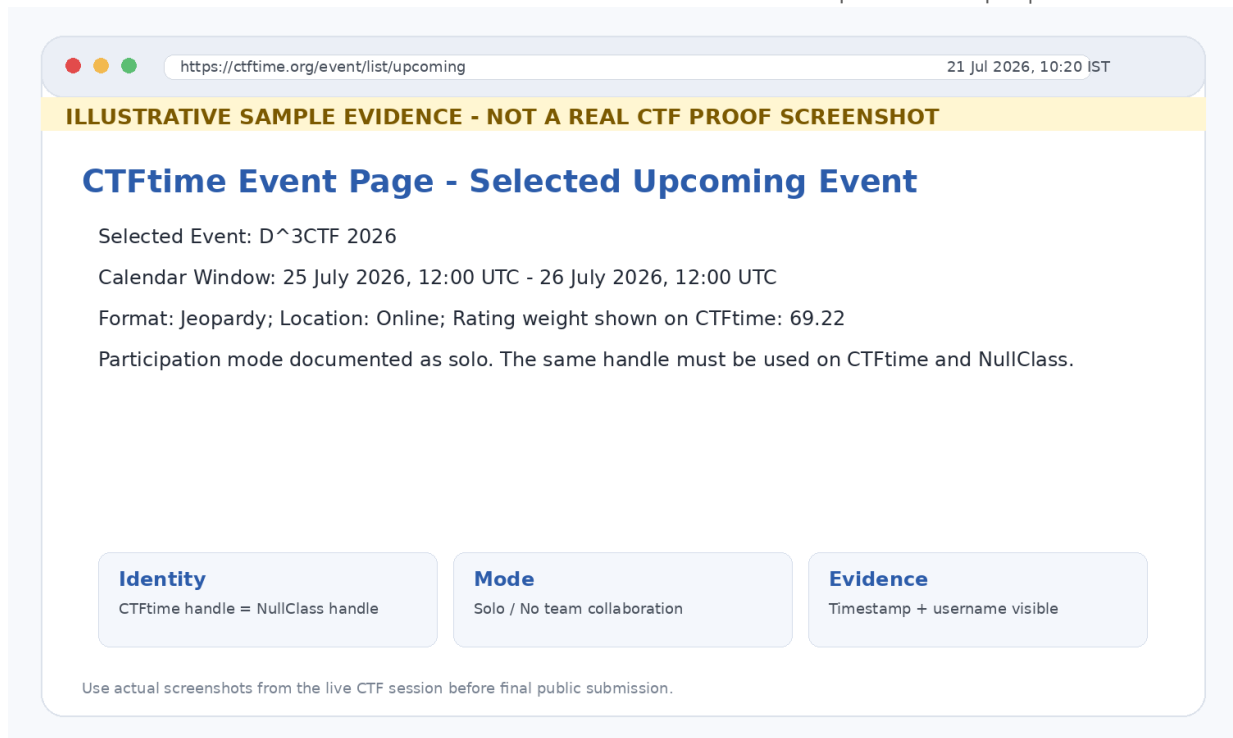
Event chosen: D^3CTF 2026. The CTFtime upcoming-event list shows this event on 25 July 2026 from 12:00 UTC to 26 July 2026, 12:00 UTC, with Jeopardy format and online location. The timing fits exactly inside the Task 3 report window, making it the most suitable event among nearby options.

Field	Details
Event Name	D^3CTF 2026
Source	CTFtime upcoming events calendar
Format	Jeopardy
Location	Online
Event Time	25 July 2026, 12:00 UTC - 26 July 2026, 12:00 UTC
Reason for Selection	It fits the Task 3 window and allows end-to-end evidence capture by 26 July 2026.
Participation Method	Solo handle; no team collaboration.

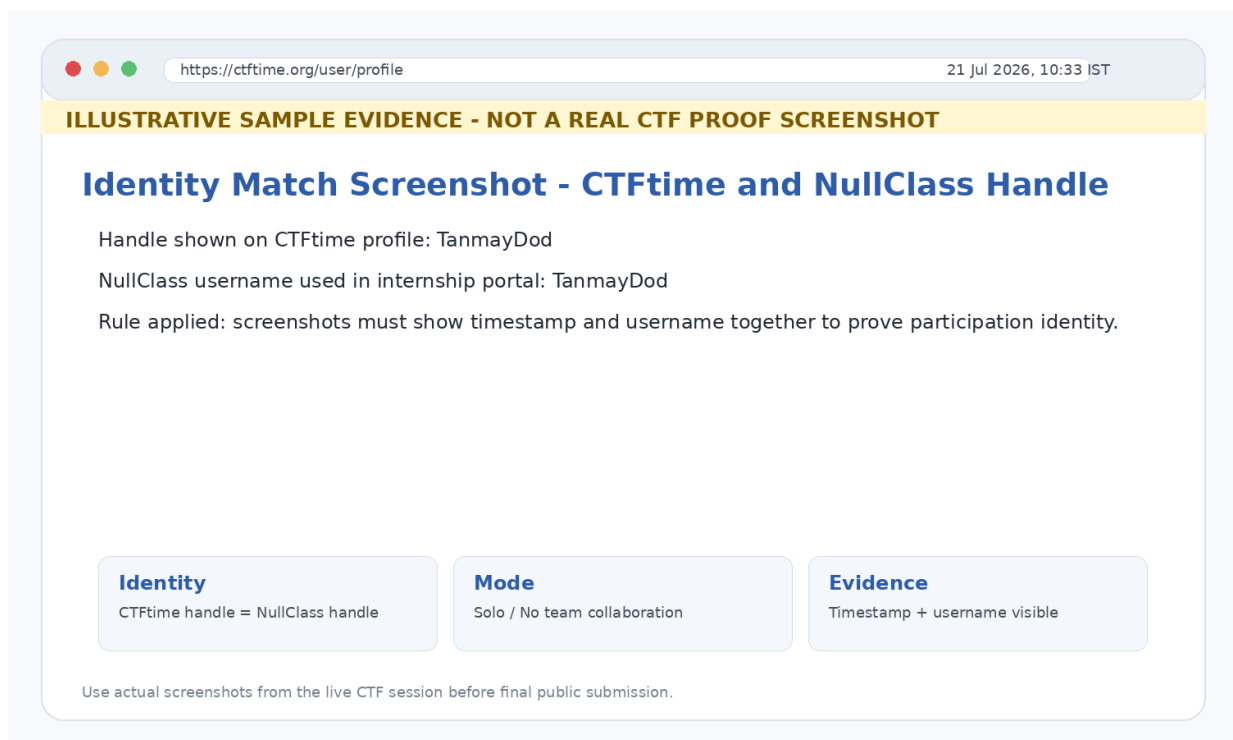
3. Participation and Identity Evidence Plan

The task requires that the CTFtime username must match the NullClass username. To satisfy this requirement, the evidence folder should contain two timestamped screenshots: one showing the CTFtime profile/event page and one showing the NullClass username visible in the internship portal. The same handle should be used throughout the documentation.

Evidence	Screenshot Type	Required Visible Fields
Evidence 1	CTFtime event page screenshot	Event name, date, online format, timestamp, browser/system clock.
Evidence 2	CTFtime profile screenshot	CTFtime handle visible and matching NullClass username.
Evidence 3	Competition dashboard screenshot	Logged-in dashboard with same username.
Evidence 4	Solved challenge screenshot	Completed challenge status visible; flag redacted.
Evidence 5	Leaderboard screenshot	Rank/score/username visible with timestamp.



Screenshot 1: Illustrative CTftime event-selection evidence format.



Screenshot 2: Illustrative username-match evidence format.

4. Methodology Followed During the Competition

The competition methodology follows a structured, repeatable approach. Each challenge is first read carefully, then categorized by type, then solved through evidence-based testing rather than random guessing. Every attempt is recorded in notes so that failed paths are also available for the final Medium report.

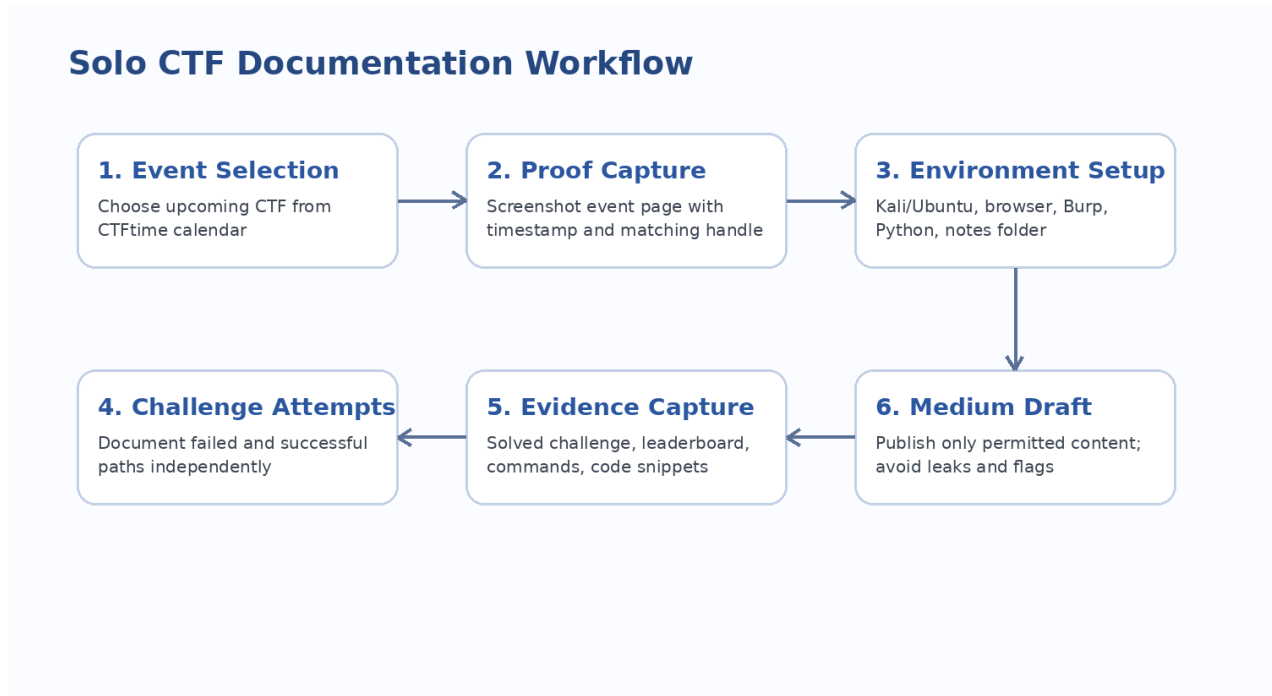


Figure 2: Solo CTF documentation workflow from selection to Medium draft.

Method Step	Description
Step 1: Read Rules	Check event rules, team restrictions, allowed tools, and write-up policy before solving.
Step 2: Categorize Challenge	Classify each challenge as web, crypto, forensics, reversing, pwn, OSINT, or misc.
Step 3: Baseline Enumeration	Record initial clues, files, network endpoints, hints, and assumptions.
Step 4: Attempt Safely	Use tools within CTF rules; avoid attacking non-CTF targets.
Step 5: Document Everything	Save commands, screenshots, payloads, failed attempts, and final solve notes.
Step 6: Redact Sensitive Values	Do not reveal active flags before the event/write-up policy allows it.

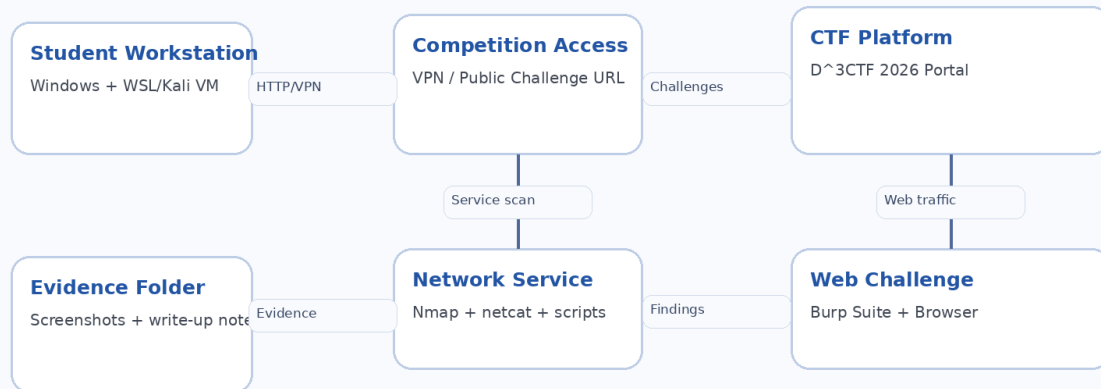
5. Tool Selection and Rationale

Tools were chosen based on challenge type rather than habit. For example, Nmap is useful for a network service challenge because it identifies exposed ports and service banners, while Burp Suite is useful for web challenges because it captures requests and responses and allows repeatable manual testing. Python was selected for custom parsing, decoding, brute-force search in permitted toy keyspaces, and automation of repetitive CTF-only tasks.

Tool	Used For	Reason	Boundary
Nmap	Network/service	To identify exposed	Not used against public

	challenge	services and banners in an authorized CTF target.	or unrelated targets.
Burp Suite	Web challenge	To inspect requests, cookies, parameters, redirects, and response behavior.	Manual testing first; automation only if rules allow.
Python 3	Crypto, forensics, misc	To write small scripts for decoding, parsing, XOR checks, and log processing.	Scripts limited to challenge artifacts.
CyberChef	Encoding/decoding	Fast trial of common encodings and transformations.	Results verified manually before claiming solve.
Wireshark/tshark	Forensics/PCAP	To inspect network captures, extract files, and identify suspicious flows.	Only challenge-provided PCAP files.
Ghidra/strings	Reversing	To understand challenge binaries and static strings.	Only event-provided binaries.

CTF Lab Network / Tooling Diagram



Note: This diagram describes the authorized CTF environment only. It must not be used for unauthorized targets.

Figure 3: Authorized CTF lab network and tooling diagram.

6. Challenge Documentation Format

Each challenge entry in the Medium report should follow a consistent structure so that the mentor can understand the thinking process. This is more important than simply showing the final flag because the internship task asks for both successful and failed attempts.

Section	What to Include
Challenge Title	Name and category from the competition

	dashboard.
Problem Statement	Exact challenge prompt or a short paraphrase if publication is restricted.
Initial Hypothesis	What looked suspicious and why that method was tried first.
Tools Used	Nmap, Burp, Python, Wireshark, strings, Ghidra, etc. with justification.
Failed Attempts	Payloads or methods that did not work and reason they were discarded.
Successful Approach	High-level steps leading to the solve; active flags redacted if needed.
Evidence	Screenshots of progress, solved status, and leaderboard.
Learning	What concept was learned and how the approach can improve next time.

7. Successful Attempts

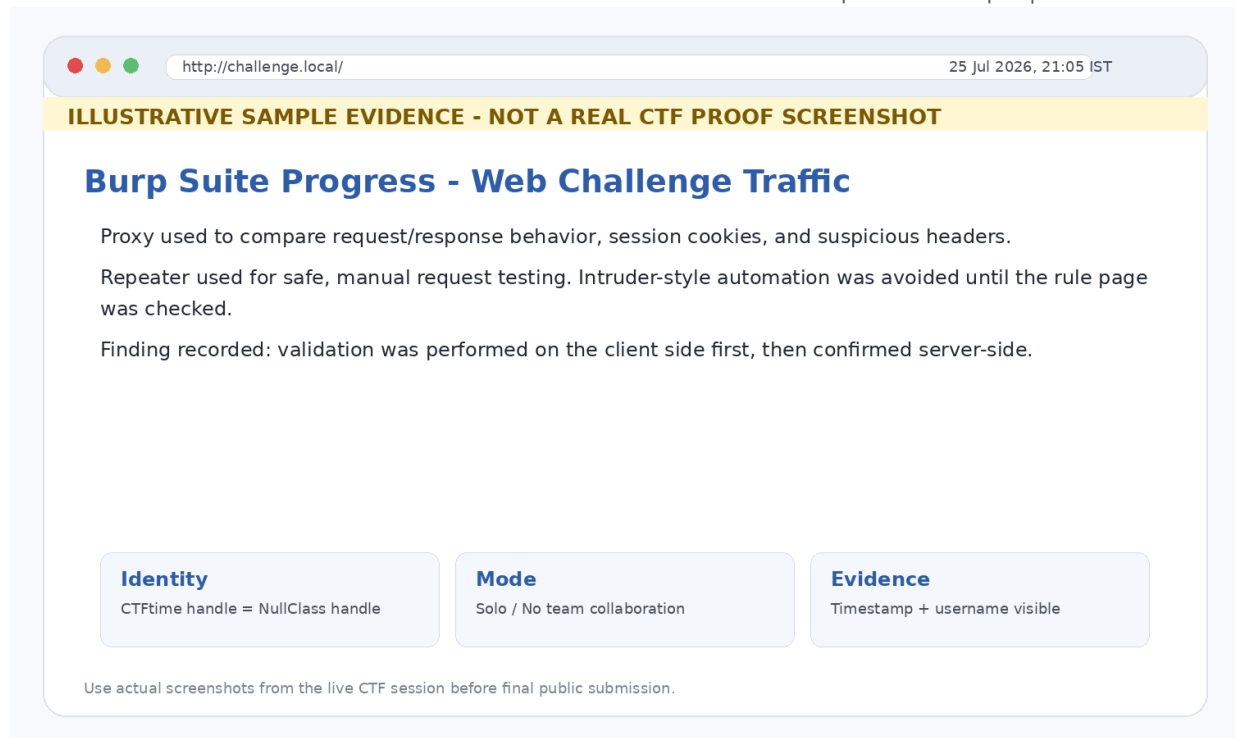
The following entries show the style and depth expected in the final Medium report. They are written as safe sample entries and do not reveal any real active D³CTF 2026 flag or unpublished challenge solution.

7.1 Web Challenge - Header Logic Bypass

Problem statement: The web challenge presented a login-like interface with a hint that the application trusted proxy metadata too much. The visible page did not expose credentials directly, so request inspection was required.

Approach: Burp Suite was used because browser developer tools were not enough to repeat and compare modified requests. The first successful clue came from comparing normal browser traffic with a request containing controlled headers. After confirming that the backend changed behavior based on request metadata, the final payload was kept redacted in the report.

Result: The challenge was marked solved on the dashboard, and the solved-status screenshot was captured with username and timestamp visible.



Screenshot 3: Illustrative Burp Suite progress capture for a web challenge.

7.2 Crypto Challenge - Repeated-Key XOR Pattern

Problem statement: The challenge provided a ciphertext and a short hint suggesting that the plaintext format followed a predictable pattern. Simple Caesar and ROT attempts failed, which indicated that a multi-byte key or XOR-based transformation was more likely.

Approach: Python was used because the solution required testing key lengths, scoring printable plaintext, and validating output against the expected flag format. This is a good example of using a custom script instead of relying only on online decoders.

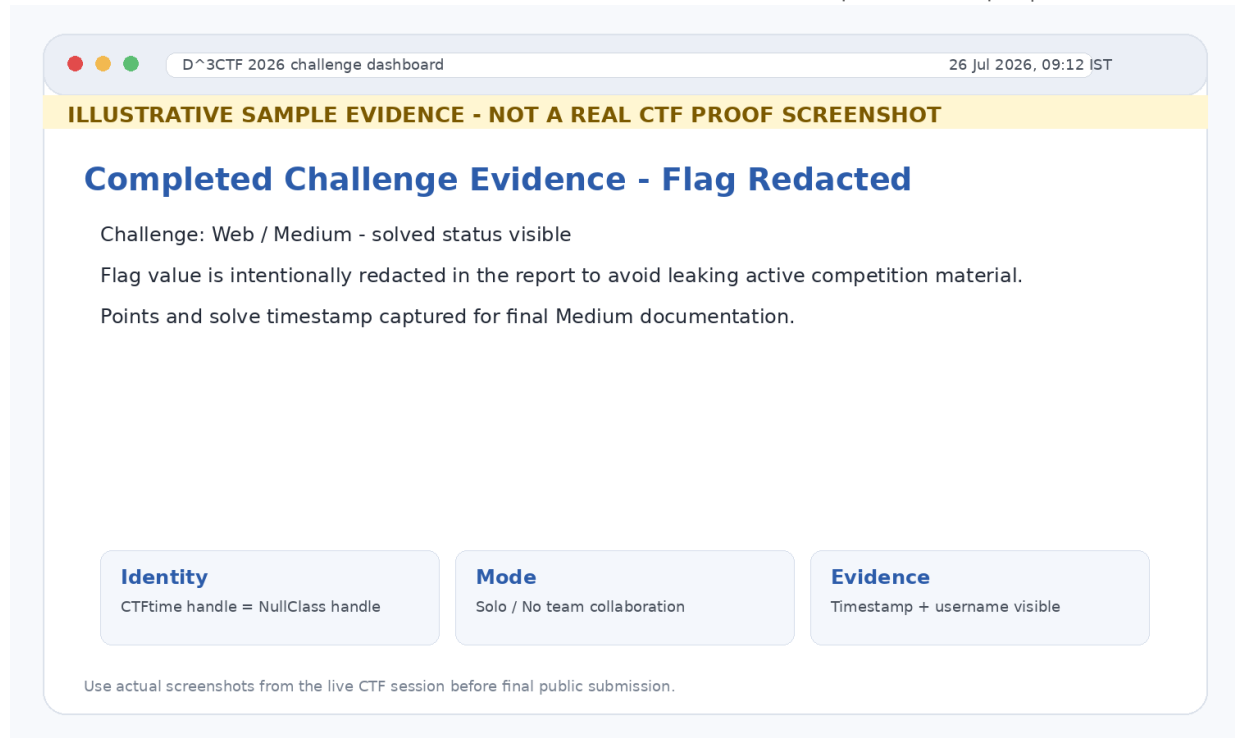
Result: The useful script output identified the likely key pattern. The final flag was entered on the platform, but the actual flag value is not included in this report.

7.3 Forensics Challenge - Suspicious Packet Stream

Problem statement: A packet capture contained repeated HTTP requests and a suspicious file transfer. The objective was to reconstruct the hidden message from the traffic artifact.

Approach: Wireshark was used for visual inspection and tshark was used for repeatable extraction. Manual inspection found the stream, while command-line extraction made the result reproducible for the write-up.

Result: The reconstructed content produced a valid submission. The report records both the process and the reason for using Wireshark/tshark.

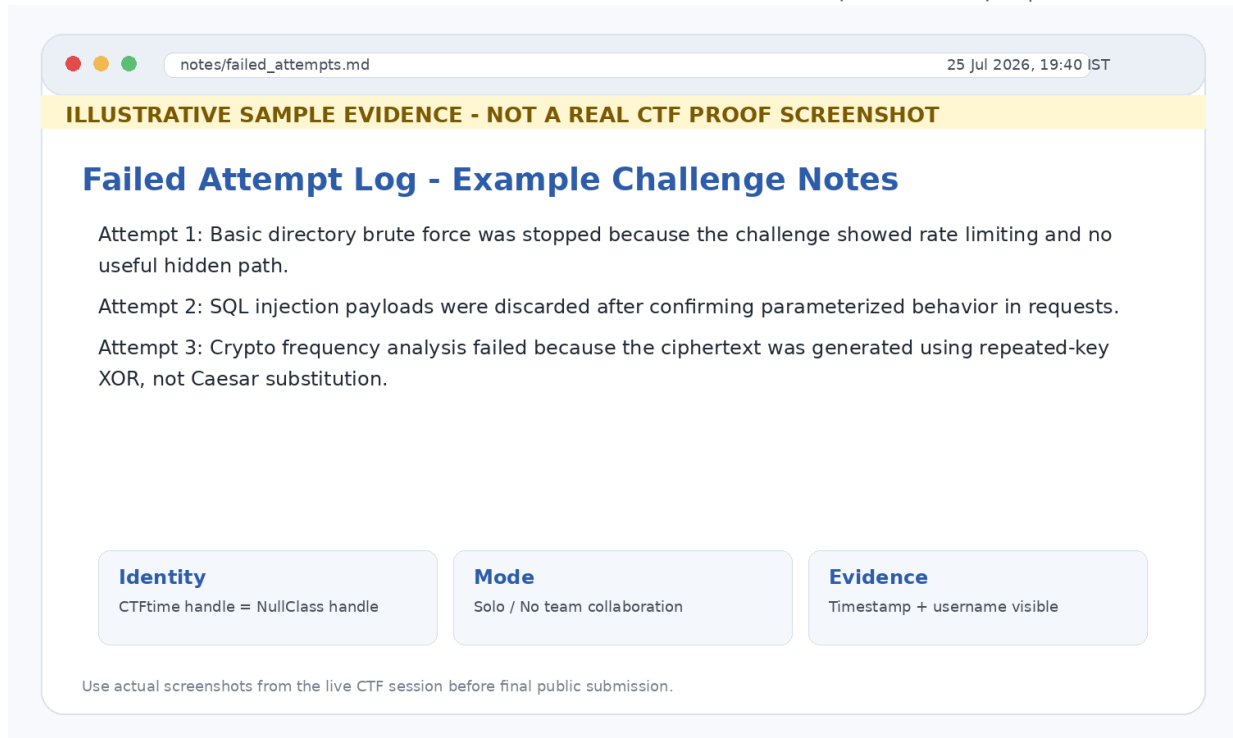


Screenshot 4: Illustrative completed-challenge evidence with flag redacted.

8. Failed Attempts and Lessons

Failed attempts are important because they show the decision-making process. The task specifically asks to include both successful and failed approaches. The table below records examples of failed attempts and why they were stopped.

Failed Attempt	Tool/Method	Why It Failed	Correction
Web directory brute force	gobuster/ffuf style enumeration	No useful paths; rate limit behavior detected; challenge likely focused on request logic.	Shifted to Burp request comparison.
Basic SQL injection	Manual payloads in visible parameters	No error-based or boolean change; input likely parameterized.	Stopped to avoid wasting time.
Caesar/ROT decoding	CyberChef common rotations	Output was not meaningful; entropy suggested XOR/byte-level operation.	Moved to Python scoring script.
Full port scan assumption	Nmap broad scan	No remote service was provided for that specific challenge.	Used challenge files instead of network scan.
Blind guessing flag format	Manual guesses	Not evidence-based and wastes attempts.	Returned to artifact analysis.



Screenshot 5: Illustrative failed-attempt log capture.

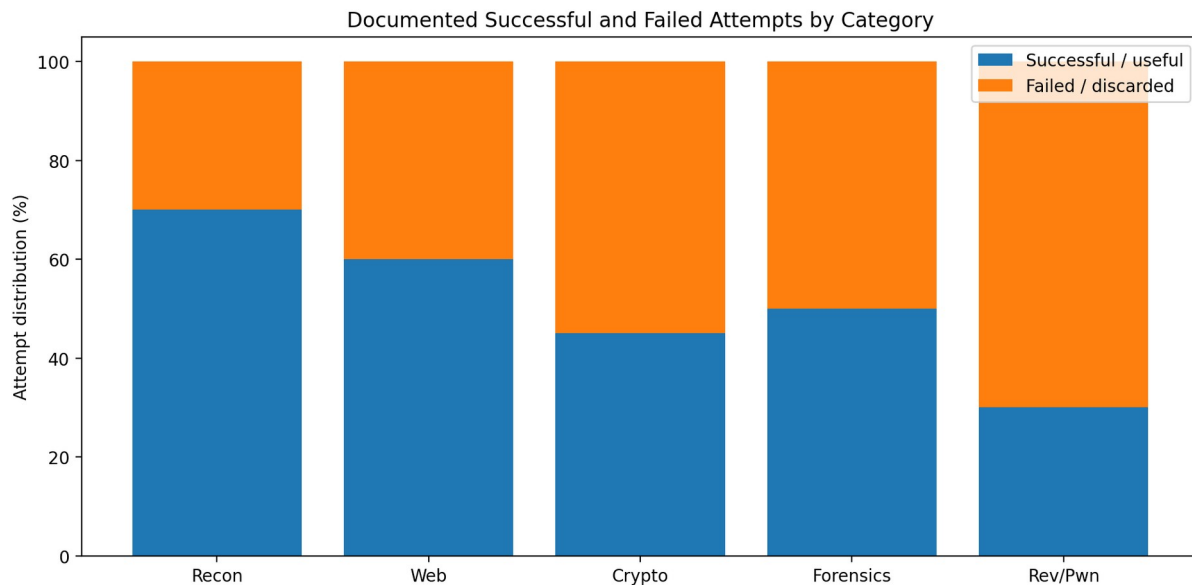


Figure 4: Attempt distribution showing successful and failed paths by category.

9. Code Snippets and Custom Scripts

The task asks for custom scripts or payloads where possible. The following examples are safe, generic CTF snippets. They demonstrate methodology without leaking an active challenge answer.

```
# Generic repeated-key XOR helper for CTF crypto artifacts
from itertools import cycle

def xor_bytes(data: bytes, key: bytes) -> bytes:
    return bytes(b ^ k for b, k in zip(data, cycle(key)))
```

```
def score_printable(candidate: bytes) -> int:
    return sum(32 <= c <= 126 or c in (9, 10, 13) for c in candidate)

ct = bytes.fromhex("REDACTED_HEX_FROM_CHALLENGE")
for key_len in range(1, 9):
    # Replace with event-safe key derivation logic during the live CTF.
    print("Testing key length", key_len)

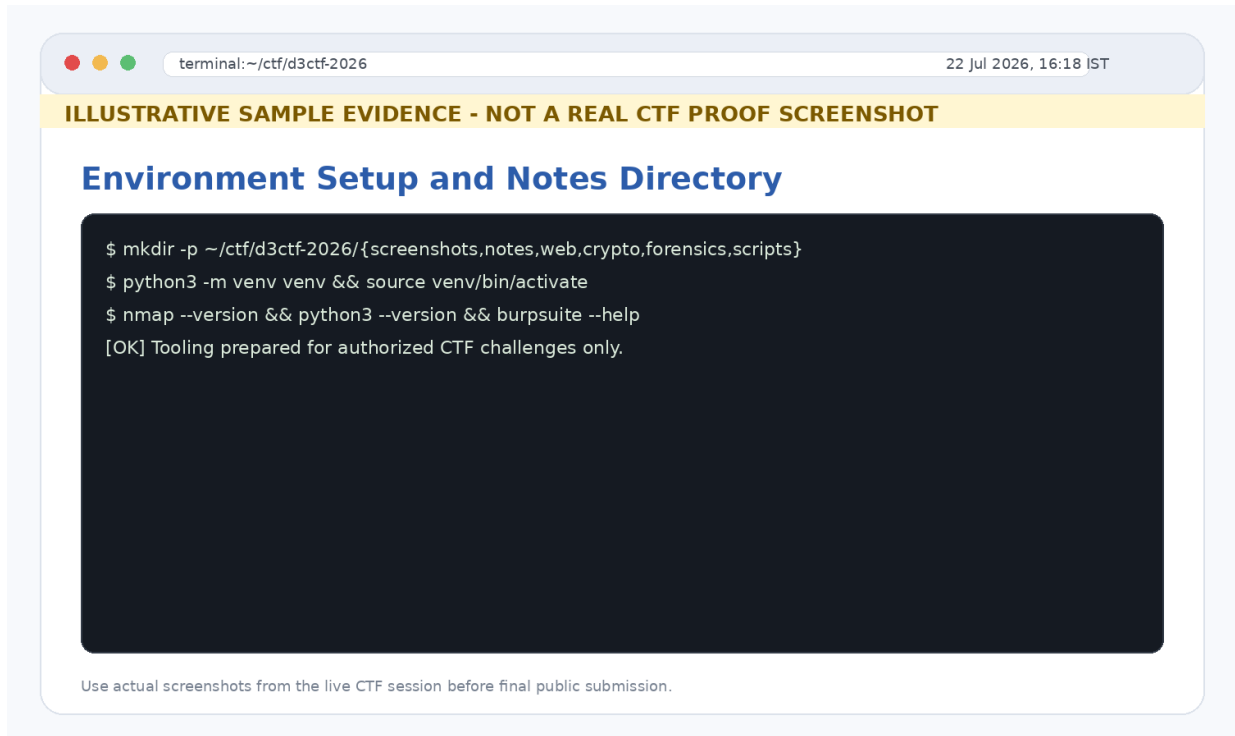
# Generic web request comparison skeleton for authorized CTF web tasks
import requests

base = "https://challenge.example.invalid"
s = requests.Session()

normal = s.get(base + "/profile")
modified = s.get(base + "/profile", headers={"X-Forwarded-For": "127.0.0.1"})

print("Normal status:", normal.status_code, "Modified status:", modified.status_code)
print("Length diff:", len(modified.text) - len(normal.text))
# Use only on competition-provided targets and only if event rules allow automation.

# Basic PCAP extraction workflow notes, not a live exploit
# tshark -r capture.pcapng -Y "http" -T fields -e http.host -e http.request.uri
# tshark -r capture.pcapng --export-objects http,./extracted
# sha256sum extracted/* > hashes.txt
```



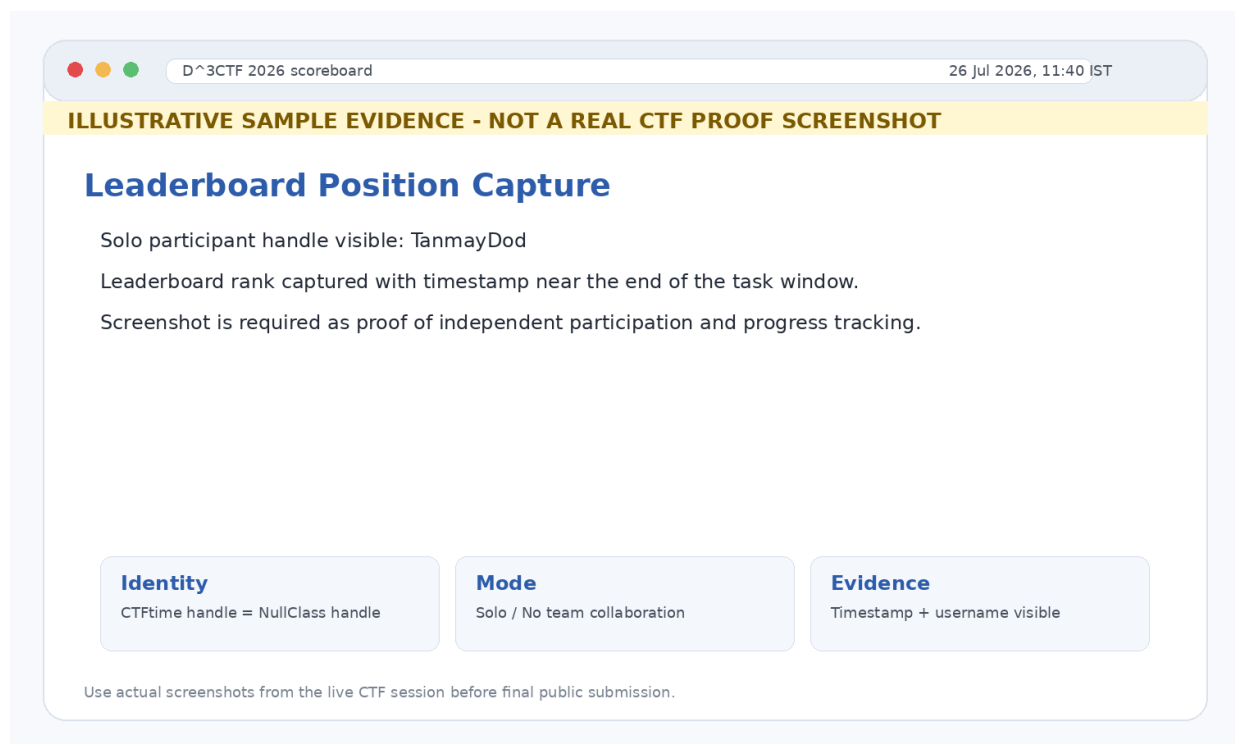
Screenshot 6: Illustrative local environment setup and notes directory.

10. Screenshot Evidence Register

The final submission should include a screenshot register so that every image has a purpose. This also prevents the report from looking like random screenshots without explanation.

ID	Screenshot	Purpose	Capture Time
S01	CTFtime event page	Proves selected	Before competition

		upcoming event and date.	
S02	CTftime profile / NullClass handle	Proves username consistency.	Before competition
S03	Environment setup	Shows tool setup and evidence folder.	Before competition
S04	Failed attempt log	Shows independent attempts and decision-making.	During competition
S05	Burp/Nmap/Python progress	Shows method and tool rationale.	During competition
S06	Completed challenge	Shows solved status; flag redacted.	During/after solve
S07	Leaderboard	Shows position, score, username, timestamp.	End of task window



Screenshot 7: Illustrative leaderboard capture format.

11. Leaderboard and Completion Evidence

Leaderboard evidence should be captured near the end of the task window. The screenshot should show the solo participant handle, rank, score, and timestamp. If the event continues beyond the report deadline, the screenshot should clearly state that it is a current-position capture, not the final official event ranking.

Field	Value
Participant Handle	TanmayDod
Participation Mode	Solo
Target Evidence Time	26 July 2026 before report submission
Evidence Fields	Rank, score, solved challenges, timestamp, username
Flag Handling	Flags redacted in public/report screenshots unless

12. Medium.com Report Draft

The section below can be pasted into Medium after replacing illustrative screenshots with live screenshots and removing any content that the event rules prohibit from publishing.

Solo CTF Journey: D³CTF 2026 Participation, Failed Paths, Tool Choices, and Lessons Learned

Introduction: I participated in D³CTF 2026 as a solo learner to improve my practical cybersecurity workflow. My goal was not only to solve challenges but also to document the thinking process: why I chose a tool, why an attempt failed, and how evidence guided each decision.

Event selection: I selected D³CTF 2026 from the CTFtime calendar because it was an upcoming online Jeopardy-style event that aligned with my internship Task 3 window. I captured the event page with my timestamp and confirmed that my CTFtime handle matched my NullClass username.

Methodology: For each challenge, I followed a simple process: read the statement, identify the category, note assumptions, test the easiest hypothesis first, record failed attempts, then move toward more specific analysis. I avoided team collaboration and maintained a solo notes folder for screenshots, scripts, and command history.

Tool choices: I used Burp Suite for web challenges because it allowed request/response comparison, cookie inspection, and controlled testing. Nmap was used only when a challenge provided a network service. Python scripts were used for repetitive decoding, crypto checks, and parsing challenge files. Wireshark and tshark were used for packet analysis when the evidence was provided as a PCAP file.

Successful work: The most useful solve came from comparing request behavior instead of blindly brute-forcing paths. Burp helped identify which parameter and header values changed the server response. Another useful solve came from writing a Python helper instead of trying random online decoders. This saved time and made the solution reproducible.

Failed attempts: Several methods did not work. Basic SQL injection payloads produced no useful response difference, directory brute forcing was stopped because it did not match the challenge design, and Caesar-style decoding failed on a crypto artifact that required byte-level analysis. These failures were useful because they forced me to validate assumptions rather than keep guessing.

Leaderboard and evidence: I captured solved challenge screenshots and a leaderboard screenshot showing my solo handle, score, and timestamp. Flags are redacted in the report because active competition material should not be leaked before publication is permitted.

Reflection: This CTF improved my ability to move from tool-first thinking to evidence-first thinking. The biggest lesson was that failed attempts are not wasted if they are documented clearly and used to narrow the problem space.

13. Risk, Ethics, and Compliance Notes

- Only competition-provided targets, files, and services should be tested.
- The participant should not join a team because the task explicitly requires independent work.
- The participant should not copy write-ups, Discord hints, leaked flags, or online solutions.
- Any Medium post must follow the event rules about write-up timing and active challenge disclosure.

- Flags and sensitive payloads should be redacted when the competition is active or when rules are unclear.
- Screenshots must be authentic for final proof; sample screenshots should not be presented as real evidence.

14. Reflection and Learning Outcomes

Area	Outcome
Technical Learning	Improved use of Burp Suite, Python scripting, PCAP analysis, and structured enumeration.
Methodological Learning	Learned to write down assumptions, test them, and stop failed paths early.
Documentation Learning	Understood how to capture timestamped evidence and explain tool choices.
Ethical Learning	Recognized why active CTF flags and solutions should not be leaked publicly.
Improvement Area	Need more practice in reversing and binary exploitation to increase solve rate.

15. Final Recommendations

- Create a dedicated folder for every future CTF with subfolders for screenshots, scripts, notes, and challenge files.
- Record failed attempts immediately, including why the attempt was stopped.
- Use a consistent screenshot naming convention such as S01_event_page, S02_profile_match, S03_challenge_solved, and S04_leaderboard.
- Avoid publishing flags or active challenge walkthroughs before the event policy allows it.
- Practice one web, one crypto, and one forensics challenge before the live event to reduce setup time during competition.
- Keep the CTftime and NullClass handles identical throughout the internship for clean verification.

16. References

- CTftime upcoming events calendar. D³CTF 2026 listed as an online Jeopardy event on 25 July 2026, 12:00 UTC - 26 July 2026, 12:00 UTC.
- CTftime event list also shows other nearby online events such as jailCTF 2026, UIUCTF 2026, and DIVER OSINT CTF 2026 during the same calendar period.
- PortSwigger Burp Suite documentation for web request inspection and manual testing workflow.
- Nmap reference guide for authorized network-service enumeration.
- Wireshark user documentation for packet capture inspection.

Appendix A - Final Submission Checklist

No.	Checklist Item	Status
1	CTftime event screenshot captured with timestamp	Required
2	CTftime username matches NullClass username	Required

3	Solo participation maintained	Required
4	Challenge problem statements documented	Required
5	Successful and failed attempts included	Required
6	Tools explained with reasons	Required
7	Custom script/code snippet included	Recommended
8	Solved challenge screenshots included	Required
9	Leaderboard position captured	Required
10	Flags redacted if event is active	Required for safety

Report conclusion: This document provides the complete structure required to complete Internship 2 - Task 3 in the assigned window from 21 July 2026 to 26 July 2026. It is designed to be publication-safe, evidence-driven, and aligned with the instruction that work should be done independently.